

Reviewer Pack — Finite Field Rank and Branching Program Width for Boolean Fu...

Entry 89d7e3ddf0de · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-10 04:08:13 UTC

Finite Field Rank and Branching Program Width for Boolean Functions

Entry ID: 89d7e3ddf0de **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-10 04:08:13 UTC

1. Conjecture

Field A (mathematical branch): Finite Field Linear Algebra **Field B** (complexity object): Branching Program Width

Statement:

For a Boolean function f on n variables, let M_f be the matrix over $\text{GF}(2)$ with rows representing variables and columns representing truth assignments, where $M_f[i][j] = f(x_j)$ if x_j has i -th variable set. The minimal branching program width for f is at least the rank of M_f over $\text{GF}(2)$.

Rationale (proposer's reasoning):

The rank of M_f captures the linear dependencies in f 's truth table. A higher rank implies more complex interactions between variables, necessitating wider branching programs to represent the function's behavior. This link could expose algebraic structure in complexity measures.

Taxonomy category: BARRINGTON_ALG (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 40c71c1a6a88f5ee

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from collections import defaultdict

def gaussian_elimination(matrix):
    rows, cols = len(matrix), len(matrix[0])
    rank = 0
    for j in range(cols):
        i_max = -1
        for i in range(rank, rows):
            if matrix[i][j] == 1:
                i_max = i
                break
        if i_max >= 0:
            matrix[rank], matrix[i_max] = matrix[i_max], matrix[rank]
            for k in range(j + 1, cols):
                matrix[rank][k] ^= matrix[i_max][k]
            rank += 1
    return rank

def bdd_width(truth_table):
    n = len(truth_table)
    if n == 0:
        return 0
    variables = list(range(n))
    width = 0
    while variables:
        var = random.choice(variables)
        variables.remove(var)
        new_vars = []
        for i in range(1 << (n - 1)):
            x = [bool((i >> j) & 1) for j in range(n)]
            if truth_table[i][var] != truth_table[2 * i][var]:
                new_vars.append(var)
                break
        width += 1
    return width
```

```

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 10
    M_f = [[random.randint(0, 1) for _ in range(2**n)] for _ in range(n)]
    rank = gaussian_elimination(M_f)
    bdd_width_value = bdd_width(M_f)
    conjecture_holds = rank <= bdd_width_value
    counterexample = "" if conjecture_holds else "bdd_width < rank"
    return {
        "metric_name": "rank vs. BDD width",
        "metric_value": rank,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    std_rank = math.sqrt(sum((r["metric_value"] - mean_rank)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"bdd_width < rank\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

TRIAL: {'metric_name': 'rank vs. BDD width', 'metric_value': 10, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'rank vs. BDD width', 'metric_value': 10, 'instances_tested': 1, 'conjecture_holds': True}

```

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b46512cb.py", line 72, in <module>
    result = run_trial(seed)
            ~~~~~^~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b46512cb.py", line 56, in run_trial
    bdd_width_value = bdd_width(M_f)
                      ~~~~~^~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b46512cb.py", line 45, in bdd_width
    if truth_table[i][var] != truth_table[2 * i][var]:

```

~~~~~^~~~~~  
IndexError: list index out of range

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

Test crashed with IndexError during BDD width calculation, preventing reliable results |  
next: Fix the index out-of-range error in bdd\_width function and re-run experiments

## 11. Audit log (LLM calls)

**Total LLM calls:** 7

| # | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|---|-----------------|---------------|------------------|-----------|-----|--------------|
| 1 | propose         | ollama_remote | qwen3:8b         | 0         | 0   | 45645        |
| 2 | preregistration | ollama_remote | qwen3:8b         | 0         | 0   | 23878        |
| 3 | novelty         | ollama_remote | qwen3:8b         | 0         | 0   | 20496        |
| 4 | novelty         | ollama_remote | qwen3:8b         | 0         | 0   | 15572        |
| 5 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 13114        |
| 6 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 8179         |
| 7 | judge           | ollama_remote | qwen3:8b         | 0         | 0   | 16390        |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 143274 ms total latency. Provider mix: {'ollama\_remote': 7}

*(full prompt+response transcripts available in research/audit/89d7e3ddf0de.jsonl)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/89d7e3ddf0de.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** research/pvsnp\_sandbox/test\_\*.py (per-cycle)

**Replay tarball:** research/replay/89d7e3ddf0de.tar.gz (if generated)